

Chapter 3: Components

1. Overview

A component is the fundamental UI building block in Angular. It is a TypeScript class decorated with `@Component`, paired with a template (HTML), optional styles (CSS), and metadata that tells Angular how to create, render, and manage instances of it inside the DOM.

Conceptually, a component is a **directive with a view**. Angular has three directive types — components, structural directives, and attribute directives — and a component is the only one that owns a template and a piece of rendered DOM. Everything else (data binding, dependency injection, change detection, lifecycle hooks) hangs off this basic unit.

Why components matter for interviews: they are the surface where almost every other Angular concept intersects — DI (constructor injection into a component), change detection (`OnPush` vs `Default` at the component level), rendering (view encapsulation, `ng-content`), and the component tree (parent/child communication via `@Input` / `@Output`). Interviewers use "explain a component" as a springboard into all of these. This chapter focuses narrowly on the component's own contract: decorator, metadata, selector matching, encapsulation, host bindings, styling, projection, and the class/template relationship — deferring `@Input` / `@Output`, lifecycle hooks, and change detection internals to their own chapters (though they're referenced here where necessary for context).

2. Core Concepts

2.1 The `@Component` Decorator

`@Component` is a TypeScript decorator — a function that runs at class-definition time and attaches metadata to the class via `Reflect.defineMetadata` (using the `reflect-metadata`-like annotations Angular's compiler consumes) or, with the modern Ivy compiler, compiles directly into static properties on the class (`ɵcmp`). It does **not** change the class's runtime behavior by itself; it registers a `ComponentDef` that Angular's renderer uses to know how to instantiate and render the class.

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-user-card',
  templateUrl: './user-card.component.html',
  styleUrls: ['./user-card.component.scss'],
})
export class UserCardComponent {
  name = 'Ada Lovelace';
}
```

Under the hood (Ivy), the compiler generates a static field on the class:

```

UserCardComponent.ɵcmp = defineComponent({
  type: UserCardComponent,
  selectors: [['app-user-card']],
  decls: /* number of DOM/template nodes */,
  vars: /* number of binding slots */,
  template: function UserCardComponent_Template(rf, ctx) { /* ... */ },
  encapsulation: 2 /* Emulated */
});

```

A class is *only* a component to Angular because it carries this compiled definition — `@Component` without a matching template/selector setup is meaningless metadata; it is the compiler's job (via `ngc` /AOT or JIT) to turn the decorator's object literal into this runtime definition.

2.2 Metadata Properties — The Full Surface

The `ComponentDecorator` metadata object accepts (this is effectively `Directive` metadata plus component-only additions):

Property	Purpose
<code>selector</code>	CSS-like selector Angular uses to match this component to elements in a template.
<code>template</code> / <code>templateUrl</code>	Inline or external HTML template. Mutually exclusive.
<code>styles</code> / <code>styleUrls</code> / <code>styleUrl</code> (v16+ single-file)	Inline or external CSS.
<code>encapsulation</code>	<code>Emulated</code> (default), <code>ShadowDom</code> , or <code>None</code> .
<code>changeDetection</code>	<code>ChangeDetectionStrategy.Default</code> or <code>OnPush</code> .
<code>providers</code>	DI providers scoped to this component's injector and its view.
<code>viewProviders</code>	DI providers visible only to the component's <i>view</i> (child view), not to projected content or <code>providers</code> consumers via content children.
<code>host</code>	Object (or now often replaced by <code>@HostBinding</code> / <code>@HostListener</code> , but still valid) to bind host element properties, attributes, classes, styles, and listen to host events.
<code>inputs</code> / <code>outputs</code>	Array-of-strings alternative to <code>@Input()</code> / <code>@Output()</code> decorators, e.g. <code>inputs: ['label: displayLabel']</code> .
<code>exportAs</code>	Name(s) under which the component instance can be referenced via a template reference variable, e.g. <code>#foo="exportAsName"</code> .
<code>animations</code>	Array of Angular animation trigger definitions.
<code>standalone</code>	(Angular 14+; default <code>true</code> since Angular 19) Whether the component declares its own dependencies via <code>imports</code> instead of belonging to an <code>NgModule</code> .

Property	Purpose
<code>imports</code>	(standalone only) Directives/components/pipes/modules this component's template depends on.
<code>changeDetection</code> , <code>preserveWhitespaces</code> , <code>interpolation</code>	Compiler-level tuning knobs.
<code>queries</code>	Legacy way to declare <code>@ViewChild</code> / <code>@ContentChild</code> via metadata instead of decorators.

Key exam-style fact: `@Component` **extends** `@Directive's metadata` — every component is a directive, so anything a `@Directive` can declare (`selector`, `inputs`, `outputs`, `host`, `providers`, `exportAs`, `standalone`) a component can declare too, plus the view-related properties (`template*`, `style*`, `encapsulation`, `animations`, `viewProviders`, `changeDetection`).

2.3 Selectors — Matching Rules

A selector tells Angular's compiler which elements in a template should be upgraded into instances of this component. Angular supports a constrained subset of CSS selector syntax:

- **Element selector:** `'app-root'` → matches `<app-root>`.
- **Attribute selector:** `'[appHighlight]'` → matches any element with that attribute, regardless of tag. Common for directives, rare (but legal) for components.
- **Class selector:** `'.my-class'` → matches elements with that class. Rarely used for components (ambiguous, styling collision).
- **Combinators:** `'button[app-button]'` (element + attribute), `':not(...)'` is *not* generally supported the way plain CSS supports it, but Angular does support a limited `:not()` pselect on some attribute directives.
- **Multiple selectors:** comma-separated, e.g. `'app-foo, [app-foo]'` — matches either.

Rules and gotchas:

1. **Only one component can match a given host element.** Two components cannot claim the same element (this *will* throw at compile time: "multiple components match node"). Multiple **directives**, however, can match the same element.
2. Selectors are resolved at **compile time**, not runtime — you cannot dynamically alter which selector a class responds to.
3. Angular does not support arbitrary descendant/combinator CSS selectors (`app-foo app-bar` meaning nested elements) the way real CSS does — components match elements, not DOM position.
4. Attribute selectors can carry a required value, e.g. `'button[type=submit]'`, matching only `<button type="submit">`.
5. A component **without a matching element** anywhere in a template is simply never instantiated through structural matching — it can still be created dynamically via `ViewContainerRef.createComponent()` or `ComponentFactoryResolver` (deprecated)/dynamic APIs, bypassing selector matching entirely.

2.4 View Encapsulation

Encapsulation controls whether a component's CSS leaks out to the rest of the page/other components, and whether outside CSS leaks in.

- **ViewEncapsulation.Emulated** (default): Angular rewrites your CSS and adds unique attributes (`_ngcontent-xxx`) to every element the component's template renders, and rewrites each selector in your stylesheet to include a matching attribute selector (`_ngcontent-xxx`). This *emulates* scoping without using real Shadow DOM. Global styles can still bleed in (because there's no real style boundary — just attribute-scoped selectors) but sibling components' styles don't affect this one, and vice versa, **for selectors written normally**. `::ng-deep` (deprecated but still functional) can pierce this to intentionally target descendant/projected content.
- **ViewEncapsulation.ShadowDom**: Angular attaches a genuine open Shadow DOM root to the host element (`element.attachShadow({ mode: 'open' })`) and moves styles inside it. True browser-level style and DOM (mostly) isolation — global CSS cannot reach in, and this component's CSS cannot leak out. Downsides: `:host-context()`-style global theming becomes harder, some third-party CSS frameworks assume no shadow boundary, and each shadow root duplicates style computation.
- **ViewEncapsulation.None**: No scoping at all. The component's CSS is hoisted to the document (or, in Ivy, kept but with no scoping attributes) and behaves like global CSS — it affects and is affected by everything.

```
@Component({
  selector: 'app-badge',
  template: `<span class="badge">New</span>`,
  styles: [`.badge { background: gold; }`],
  encapsulation: ViewEncapsulation.None, // .badge now leaks globally
})
export class BadgeComponent {}
```

2.5 Component Class vs Template Relationship

The **class** is the component's *controller/model* — it holds state (properties), behavior (methods), injected dependencies, and lifecycle hooks. The **template** is a *view* that binds to that class instance through a well-defined, one-directional-plus-events contract:

- **Interpolation** `{{ expr }}` and **property binding** `[prop]="expr"` read values *from* the class into the DOM.
- **Event binding** `(event)="handler($event)"` calls *into* the class from the DOM.
- **Two-way binding** `[(ngModel)]="value"` is sugar combining both directions.
- Template expressions are evaluated in the **context of the component instance** — `this` inside a template expression implicitly refers to the component instance; you cannot access arbitrary global scope or `window` unless explicitly exposed as a class member.
- Templates are **compiled**, not interpreted at runtime dynamically like an eval — the Ivy compiler turns each template into a `template()` function containing explicit instruction calls (`ɵɵelementStart`, `ɵɵtext`, `ɵɵproperty`, `ɵɵlistener`, etc.), invoked during change detection.

This separation means a component class has **zero direct DOM references** by default — no `document.getElementById`, no imperative DOM manipulation baked into the class. Direct DOM access, when required, goes through `ElementRef`, `Renderer2`, `ViewChild` — never a raw global DOM query — to keep the component server-render-safe and testable.

2.6 Host Bindings and Host Listeners

The "host element" is the actual DOM element the component's selector matched (e.g., the literal `<app-user-card>` tag). A component's *template* renders **inside** that host element; the host element itself is controlled through *host bindings*.

Two mechanisms, functionally equivalent, exist:

A. Decorator style (most common, encouraged for readability):

```
@Component({ selector: 'app-toggle', template: `...` })
export class ToggleComponent {
  @HostBinding('class.active') isActive = false;
  @HostBinding('attr.aria-pressed') get ariaPressed() { return String(this.isActive); }
  @HostBinding('style.cursor') cursor = 'pointer';

  @HostListener('click')
  onClick() {
    this.isActive = !this.isActive;
  }

  @HostListener('window:resize', ['$event'])
  onResize(event: Event) {
    // react to viewport resize
  }
}
```

B. Metadata `host` object style (compiles to the same instructions, some teams prefer it for co-location and it's required for the newer `host` object binding to signals-based APIs):

```
@Component({
  selector: 'app-toggle',
  template: `...`,
  host: {
    '[class.active]': 'isActive',
    '[attr.aria-pressed]': 'isActive',
    '(click)': 'onClick()',
    '(window:resize)': 'onResize($event)',
  },
})
export class ToggleComponent {
  isActive = false;
  onClick() { this.isActive = !this.isActive; }
  onResize(event: Event) {}
}
```

Rules:

- `@HostBinding('class.x')`, `('attr.x')`, `('style.x')`, or a bare property name (`'id'`, `'tabIndex'`) binds a **DOM property/attribute/class/style of the host element**, not the template.
- `@HostListener('eventName', ['$event'])` attaches a listener on the host element (or a global target using `window:`, `document:`, `body:` prefixes).
- Host bindings participate in normal change detection — they are re-evaluated every CD cycle like any other binding.
- Since Angular 14.3/15+, `host` metadata (and `@HostBinding` / `@HostListener`) also works seamlessly alongside **directive composition API** (`hostDirectives`), letting a component *inherit* another directive's host bindings.

2.7 Component Styling

- **Style sources:** `styles: [...]` (inline array of strings, or since Angular 17.3 a single template-literal string works too), `styleUrls: [...]`, or `styleUrl` (Angular 16+, singular, one file).
- Styles are **scoped per component** according to `encapsulation` (see 2.4).
- `:host` selector targets the host element itself from inside the component's own stylesheet: `:host { display: block; }`.
- `:host()` with a parameter, e.g. `:host(.active) { border: 2px solid green; }`, applies conditionally based on a class/attribute present on the host.
- `:host-context(.theme-dark)` matches when an ancestor (anywhere up the DOM, not just direct parent) carries the given class — useful for theming.
- `::ng-deep` (deprecated, no official replacement yet, but still widely used and functioning as of Angular 19/20) disables encapsulation for the *rest* of a selector chain, letting you style projected/child content from a parent's stylesheet. Angular team has signaled intent to remove eventually but hasn't shipped a replacement, so it remains an accepted (if discouraged) pattern.
- CSS custom properties (`--my-var`) cross encapsulation boundaries by design (they're inherited like any CSS custom property), making them the recommended way to theme components without `::ng-deep`.
- Global styles (`angular.json` → `styles` array, or `styles.css`) are not scoped at all and are the correct place for true resets/typography.

2.8 ng-content — Content Projection Basics

`ng-content` lets a component accept and render **arbitrary markup supplied by its consumer**, similar in spirit to `children` / `slot` in other frameworks or native `<slot>` in Web Components.

```
<!-- card.component.html -->
<div class="card">
  <header><ng-content select="[card-title]"></ng-content></header>
  <section><ng-content></ng-content></section>
  <footer><ng-content select="[card-footer]"></ng-content></footer>
</div>
```

```

<!-- usage -->
<app-card>
  <h2 card-title>Invoice #1029</h2>
  <p>Total due: $420.00</p>
  <button card-footer>Pay now</button>
</app-card>

```

Rules:

- **Single (unnamed) slot:** a bare `<ng-content></ng-content>` catches all projected nodes not claimed by a `select`-qualified slot.
- **Named/multi-slot projection:** `select="..."` accepts a CSS selector (tag name, attribute, or class) to route matching projected content into that specific slot.
- Content is projected **once**, at compile time of the parent's view creation — `ng-content` is a placeholder, not a live re-renderable container; it does not support `*ngIf` / `*ngFor` directly on itself (wrap it in an `<ng-container>` if conditional projection is needed).
- Projected content's change detection and bindings are evaluated in the context of the **consumer/parent component**, not the component doing the projecting — a subtlety interviewers love to probe (see Edge Cases).
- `<ng-content>` is *not* the same as `ViewChild` / `ContentChild` querying — projection is purely structural placement; querying is how the class inspects that projected content.

2.9 Component API Surface — Summary

The "public contract" of a component, as seen by:

- **The template compiler:** `selector`, `inputs` / `outputs` (incl. decorator-based `@Input` / `@Output` — covered in depth in the Data Binding chapter), `exportAs`.
- **The DI system:** `providers`, `viewProviders`, constructor parameter types.
- **The DOM:** `host` bindings/listeners, `encapsulation`, styles.
- **Consumers via templates:** template reference variables (`#ref`, using `exportAs` if not the default component instance), `ng-content` slot contract (which `select` attributes exist).
- **Consumers via code:** public methods/properties on the class accessed through `@ViewChild` / `@ContentChild`.

3. Code Examples

3.1 A fully-specified standalone component

```

import { Component, HostBinding, HostListener, ViewEncapsulation, ChangeDetectionStrategy }
from '@angular/core';
import { CommonModule } from '@angular/common';

@Component({
  selector: 'app-rating-star, [app-rating-star]',
  standalone: true,
  imports: [CommonModule],

```

```

changeDetection: ChangeDetectionStrategy.OnPush,
encapsulation: ViewEncapsulation.Emulated,
exportAs: 'ratingStar',
template: `
  <span class="star" [class.filled]="filled">
    <ng-content></ng-content>
  </span>
`,
styles: [
  :host { display: inline-block; cursor: pointer; }
  .star.filled { color: gold; }
  :host-context(.theme-dark) .star { color: #ccc; }
],
host: {
  'role': 'button',
  '[attr.aria-pressed]': 'filled',
  '(click)': 'toggle()',
  '(keydown.enter)': 'toggle()',
},
})
export class RatingStarComponent {
  filled = false;

  @HostBinding('class.disabled')
  disabled = false;

  @HostListener('mouseenter')
  onHover() {
    if (!this.disabled) this.filled = true;
  }

  toggle() {
    if (!this.disabled) this.filled = !this.filled;
  }
}

```

Usage with a template reference variable relying on `exportAs`:

```

<app-rating-star #star="ratingStar">★</app-rating-star>
<button (click)="star.toggle()">Toggle externally</button>

```

3.2 Multi-slot `ng-content` with a fallback

```

@Component({
  selector: 'app-panel',
  standalone: true,
  template: `
    <div class="panel">
      <div class="panel-header">
        <ng-content select="[panel-title]"></ng-content>
      </div>

```

```

    <div class="panel-body">
      <ng-content></ng-content>
    </div>
    @if (!hasActions) {
      <div class="panel-actions-default">No actions</div>
    }
    <div class="panel-actions">
      <ng-content select="[panel-actions]"></ng-content>
    </div>
  </div>
  `,
},
})
export class PanelComponent {
  hasActions = false; // could be set via ContentChild presence check in ngAfterContentInit
}

```

3.3 Host bindings driving conditional styling without touching the DOM directly

```

@Component({
  selector: 'tr[app-highlight-row]',
  standalone: true,
  template: `<ng-content></ng-content>`,
  host: {
    '[class.row-error]': 'hasError',
    '[class.row-warning]': 'hasWarning && !hasError',
  },
})
export class HighlightRowComponent {
  @Input() hasError = false;
  @Input() hasWarning = false;
}

```

This attaches directly to a native `<tr>` element (`selector: 'tr[app-highlight-row]'`), demonstrating that components need not introduce a custom tag — they can decorate an existing semantic element, which is common for table-row or form-control wrapper components.

4. Internal Working

4.1 From Decorator to Compiled Definition

1. **Parse time:** The Angular compiler (`@angular/compiler-cli` , run via the Angular CLI's `ngc/esbuild-` based builder) statically analyzes the `@Component({...})` object literal. It must be a *statically analyzable* object — you cannot compute `selector` at runtime from a variable that isn't a literal/const-evaluable expression, because AOT needs to resolve this at build time.
2. **Template parsing:** `templateurl / template` content is parsed into an HTML AST, then Angular's template compiler lowers bindings, structural directives, and control flow (`@if / @for`) into a sequence of low-level **instructions**.
3. **Codegen:** The class receives a static `ɵcmp` field (via `defineComponent()`), containing:

- `selectors`: parsed selector matcher data structure (an array-based encoding Angular's selector-matching algorithm can scan quickly).
 - `decls / vars`: sizes for the view's LView data structure (slots for DOM nodes and binding values).
 - `template`: the actual render function, called with `(rf, ctx)` — `rf` (render flags) distinguishes the **creation pass** (`rf & 1`, build DOM nodes/bindings once) from the **update pass** (`rf & 2`, re-run bindings every CD cycle) so the same function serves both.
 - `hostBindings`: a similarly generated function for host property/attribute/listener setup, run against the host element.
 - `styles`: an array of (possibly encapsulation-rewritten) CSS strings.
 - `encapsulation`, `changeDetection` numeric flags.
4. **Directive/Component matching**: at template-compile time of *any other* template that uses this component's selector, the compiler consults the **selector matcher**, built from all known components/directives in scope (via `imports` for standalone, or the owning `NgModule`'s `declarations`), and emits a call to instantiate this component's factory at that DOM position.

4.2 View Encapsulation Implementation Strategies

- **Emulated**: at compile time, Angular assigns two synthetic attributes per component: `_ngcontent-c{n}` (marks elements *rendered by* this component, applied to every element in its template) and `_nghost-c{n}` (marked on the host element itself). The stylesheet is rewritten so that `.foo { color: red }` becomes `.foo[_ngcontent-c{n}] { color: red }`, and `:host` rules become `[_nghost-c{n}] { ... }`. This is pure attribute-selector scoping — no real style boundary exists, so extremely high-specificity or global selectors can still cross it, and `::ng-deep` works by simply *stripping* the scoping attribute requirement for whatever the deep combinator targets.
- **ShadowDom**: at component creation, Angular calls `hostElement.attachShadow({ mode: 'open' })` and renders the component's view inside that shadow root, injecting a `<style>` tag with the *unmodified* CSS (no attribute rewriting needed — the browser's native shadow boundary does the scoping). Angular keeps a reference so change detection still targets the right render tree; from JS, `element.shadowRoot` exposes the real Shadow DOM. This is the only strategy providing true, browser-enforced isolation, including from `::ng-deep`-style overrides.
- **None**: styles are registered globally (Angular still deduplicates identical style text application across instances) with zero scoping attributes — functionally identical to just including that CSS in a global stylesheet.

Angular's renderer abstracts over these — `Renderer2` / the Ivy renderer decides, based on the compiled `encapsulation` flag, whether to call `createElement` in the light DOM with scoping attributes or delegate to `attachShadow`.

4.3 Component Instance Creation and Tracking

- Every rendered component gets an `LView` (an array-based internal data structure) holding: the component instance itself, all DOM node references for that view, all binding "memory slots" (for dirty-checking on the next CD pass), the injector reference, `TView` (a *shared, compiled-once* template metadata structure reused across all instances of the same component — separating "what to do" (`TView`, per type) from "current state" (`LView`, per instance) is core to Ivy's efficiency).

- The whole application is a **tree of LViews** rooted at the bootstrap component; each component's `LView` is nested inside its parent's, mirroring the DOM tree. `viewRef`, exposed via `changeDetectorRef`, is essentially a handle onto one `LView`.
- Component instantiation itself: Angular calls the class's constructor via **DI-resolved arguments** (the compiler generates a `factory` function per component, e.g. `UserCardComponent.ɵfac = function(t) { return new (t || UserCardComponent)(ɵdirectiveInject(HttpClient)); }`), then runs the `hostBindings` function once (creation mode) to wire host bindings/listeners, then runs `template()` in creation mode to build the child DOM, and finally schedules the first change-detection pass (update mode) before/along with `ngOnInit`.
- **Tracking for change detection:** Ivy walks the `LView` tree depth-first each CD cycle (unless `OnPush` short-circuits an unchanged subtree that received no new `@Input` reference and no event originated within it). Because `TView` is shared, but `LView` is per-instance, Angular can create thousands of instances of the same component cheaply — only `LView`'s (state) is allocated per instance; the compiled render/host-binding functions (`TView`) are shared.
- **Destruction:** `ViewContainerRef` or structural directives calling `.clear()` / `.remove()` trigger `ngOnDestroy`, detach the `LView` from its parent, run any registered `DestroyRef` callbacks, and let the DOM nodes be garbage collected once removed and unreferenced.

5. Edge Cases & Gotchas

1. **Two components can't share a selector on one element** — Angular throws a compile-time error ("More than one component matched on this element"), but a component *and* several directives can coexist on the same element without conflict.
2. `ViewEncapsulation.Emulated` **is not real isolation**. A sufficiently generic global selector (`div { color: red; }` in a global stylesheet) still overrides emulated component styles unless the component's own selector has equal-or-higher specificity, because scoping only adds an attribute selector — it doesn't create a genuine cascade boundary like Shadow DOM does.
3. **Projected content's bindings evaluate in the *parent's* context, not the child's**. If `app-card` projects `<p>{{ title }}</p>`, `title` resolves against whatever component wrote that markup — the *consumer* — not `CardComponent`. This surprises people who expect `ng-content` to behave like a child template.
4. `ng-content` **is not reactive/conditional by default**. You can't put `*ngIf` directly on `<ng-content>` in older syntax without wrapping in `<ng-container *ngIf="cond"><ng-content></ng-content></ng-container>`; with new control flow, `@if` around it works the same way — but the underlying projected nodes are still fixed at the parent's compile time, only their *visibility* toggles.
5. **Content not matching any `select` is silently dropped**, unless there's also a catch-all unnamed `<ng-content>` present. Forgetting the unnamed slot is a common bug — content authors are confused why some markup "disappears."
6. `host` **metadata object and `@HostBinding` / `@HostListener` can both target the same property and silently conflict/override** if used inconsistently across inherited classes — mixing styles across a class hierarchy is a subtle source of "why isn't my host binding working" bugs.
7. `@HostBinding('style.foo')` / `class.foo` **on the host element competes with Angular's own internal `[ngClass]` / `[ngStyle]` bindings if you also apply those** on the same host tag externally — last-write-wins per binding priority order can be non-obvious.
8. `:host-context()` **looks upward through the *actual DOM ancestor chain*, not the component tree**

— if the component is dynamically inserted somewhere unexpected (e.g., via `ViewContainerRef` into a different DOM location, or content-projected into a different physical parent), `:host-context` picks up ancestors from wherever it physically landed.

9. `::ng-deep` **deprecation status:** it's deprecated and Angular has stated intent to remove it eventually, but as of current stable releases there is *no direct replacement* other than `ViewEncapsulation.None` on a wrapper or CSS custom properties — interviewers sometimes probe whether candidates know it's still functional but discouraged, not simply "removed."
10. **Selector `'[attr]'` component selectors can accidentally match unrelated third-party elements** carrying the same attribute name if the attribute string isn't sufficiently namespaced (e.g., a generic `[title]` selector clashing with the native `title` attribute).
11. **Dynamically created components (`ViewContainerRef.createComponent()`) bypass selector matching entirely** — you can instantiate a component whose selector doesn't even appear anywhere in any template; selectors only matter for *template-based* discovery.
12. **Standalone components still need explicit `imports`** for every directive/pipe/component their template references — forgetting to add a used child component to `imports` produces a runtime "not a known element" template error, the standalone-era equivalent of forgetting an `NgModule` declaration.
13. **Style file glob mistakes:** `styleur1s` accepts multiple files whose rules are concatenated in array order — later files can unintentionally override earlier ones if both define the same selector.
14. **Shadow DOM encapsulation breaks some global concerns** — e.g., global CSS frameworks (Bootstrap classes applied from outside), `<style>`-based print stylesheets, or third-party widgets expecting to reach into your markup via plain CSS, none of which cross a real shadow boundary.

6. Interview Questions & Answers

Q1. What is the relationship between `@Component` and `@Directive`?

A component is a directive with an attached view (template). `@Component`'s metadata is a superset of `@Directive`'s metadata — everything a directive can declare (selector, inputs, outputs, providers, host bindings, exportAs) a component can declare too, plus template/style/encapsulation/animation/viewProviders/changeDetection properties that only make sense when there's a view to render.

Q2. Can two components match the same host element?

No. Angular's compiler enforces that at most one component can claim a given element via selector matching; attempting to have two components with overlapping selectors match the same tag is a compile-time error. Multiple attribute *directives*, however, can coexist on the same element without conflict.

Interviewer intent: Checks whether the candidate confuses "directive" with "component" and understands that the one-component-per-element rule is specifically about the *view-owning* directive type, not directives in general.

Q3. Explain the three `ViewEncapsulation` modes and how each is implemented.

- `Emulated` (default): Angular rewrites the component's CSS selectors and DOM elements with unique auto-generated attributes (`_ngcontent-*` / `_ngghost-*`), simulating scoping via attribute selectors — no real style boundary exists.
- `ShadowDom`: Angular attaches a genuine Shadow DOM root to the host element (`attachShadow`) and injects the unmodified CSS inside it; the browser enforces real isolation.

- `None`: styles are applied globally with no scoping at all, behaving like a plain global stylesheet.

Q4. Why can't you put `*ngIf` directly on `<ng-content>`?

`<ng-content>` is a projection placeholder, not a rendering element with its own instantiation lifecycle the way a template-backed element is — historically Angular's structural directive machinery needed an actual element/`ng-container` to attach to. The standard workaround is wrapping it: `<ng-container *ngIf="cond"><ng-content></ng-content></ng-container>` (or the modern `@if` block around it). This only toggles visibility of already-projected content; it doesn't affect when/how that content itself was created.

Q5. In whose context are expressions inside projected content evaluated?

The **consumer/parent's** context — the component that authored the projected markup — not the component doing the projecting. If `<app-card><p>{{title}}</p></app-card>` is used inside `AppComponent`, `title` resolves against `AppComponent`, regardless of what `CardComponent`'s own class defines.

Interviewer intent: This is one of the most commonly misunderstood aspects of content projection; the question filters out candidates who've only used `ng-content` superficially versus those who understand Angular's view-hierarchy binding contexts.

Q6. What's the difference between `@HostBinding` / `@HostListener` and the `host` metadata property?

They are two syntaxes producing the identical compiled `hostBindings` function on the component definition — purely a style/ergonomics choice. Decorators co-locate the binding with the property/method inline; the `host` object centralizes all host wiring in the metadata block. They can be mixed, but doing so across inheritance hierarchies can create hard-to-trace override behavior since both ultimately write to the same generated function.

Q7. What does `exportAs` do, and when is it required versus optional?

`exportAs` lets template authors capture a component/directive instance in a template reference variable using a name other than the default (which, for a component with no `exportAs`, is simply not accessible by name unless the tag itself is referenced, e.g. `#ref` without an `=name` refers to the element or, for a component, the component instance by default). It becomes necessary when multiple directives/components sit on the same element and you must disambiguate which one `#ref="xyz"` should bind to, since without an explicit name, `#ref` on an element with a component defaults to that component instance — but if there's also a directive you want to reference, you need `exportAs` on it.

Q8. Why can selectors not be dynamically computed from a runtime variable?

Angular's AOT compiler needs to statically resolve `selector` (and other metadata) at build time to generate the selector-matching data structures and wire up template compilation across the whole app. A selector computed from a non-literal, non-const-foldable expression can't be resolved during static analysis, so Angular requires metadata properties to be statically analyzable expressions (string literals, or simple const references the compiler can inline).

Q9. How does Angular decide whether a re-render is needed for a component using

`ChangeDetectionStrategy.OnPush`?

`OnPush` tells Angular's change detector to skip a component's subtree unless one of: (a) an `@Input` receives a new object reference (not just a mutated property of the same reference), (b) an event originates from within the component's own template (including host listeners), (c) an `AsyncPipe` in its template emits a new value, or (d) the component is explicitly marked dirty via `ChangeDetectorRef.markForCheck()` or a Signal read inside its template changes (with the signals-based reactivity introduced in recent Angular versions). This is orthogonal to but frequently discussed alongside component metadata since `changeDetection` lives in `@Component`'s options.

Q10. What internal data structures back a component instance at runtime (Ivy)?

Each component instance is represented by an `LView` (an array holding the component instance, DOM node references, binding state slots, and injector info), paired with a `TView` — a compiled, per-component-type (not per-instance) structure describing the template and host binding instructions. `TView` is created once and shared across every instance of that component type; only `LView` is allocated per instance, which is what makes creating many instances of the same component cheap.

Interviewer intent: Distinguishes candidates with real Ivy internals knowledge from those who've only used Angular at the API level; also sets up follow-up questions about change detection performance.

Q11. What happens if projected content matches no `select` attribute and there's no unnamed `<ng-content>` present?

That content is silently dropped — it is parsed and available to Angular internally as part of the host element's children, but never rendered anywhere, because no slot claims it. This is a common source of "my content just disappeared" bugs; the fix is adding a catch-all unnamed `<ng-content></ng-content>` if arbitrary/unclassified content should still render somewhere.

Q12. How does `:host-context()` differ from `:host()`?

`:host(.foo)` matches the host element itself when it also carries class/attribute `.foo` (i.e., a condition on the component's own host element). `:host-context(.foo)` instead looks **up the actual DOM ancestor chain** (any ancestor, not just a direct parent, and not scoped to the "component tree" — the literal DOM tree) for an element carrying `.foo`, commonly used to let a top-level theme class (e.g. on `<body>` or a root wrapper) cascade styling decisions down into deeply nested components without needing `::ng-deep` or prop-drilling a theme input everywhere.

Q13. Is a component's class allowed to directly manipulate the DOM (e.g., `document.querySelector`)?

Technically yes in the browser, but it's an anti-pattern Angular explicitly designs against: direct DOM access bypasses Angular's change detection and rendering abstraction, breaks server-side rendering (`document/window` don't exist the same way in SSR contexts) and Web Worker rendering, and creates code that can't be reasoned about via the component's declarative binding contract. The sanctioned escape hatches are `ElementRef` (with `Renderer2` for actually mutating anything, since `Renderer2` remains platform-agnostic) and `@ViewChild/@ContentChild` to reach into rendered elements/components safely.

Q14. Can a component's selector be an attribute rather than a custom element, and why would you do that?

Yes — e.g. `selector: 'tr[app-highlight-row]'` or even just `'[appSomething]'`. This lets a component "attach" its behavior/view augmentation onto a native, semantically meaningful element (like `<tr>`, `<button>`, or `<a>`) instead of forcing consumers to use a custom tag, which matters for accessibility, valid HTML nesting rules (e.g., `<tr>` must live inside `<table>/<tbody>`, so a custom `<app-row>` tag would be invalid there), and for reducing unnecessary DOM wrapper elements.

Interviewer intent: Probes whether the candidate understands selectors are a matching mechanism, not a requirement to invent new tag names, and whether they've dealt with real-world constraints like table markup validity.

7. Quick Revision Cheat Sheet

- `@Component` = `@Directive` + **view** (template, styles, encapsulation, animations, viewProviders, `changeDetection`).
- **Selectors:** element / attribute / class / comma-combined; resolved at compile time; only one component

per element, but many directives allowed.

- **Encapsulation:** `Emulated` (attribute-scoped CSS, default, not a real boundary) · `ShadowDom` (real browser isolation via `attachShadow`) · `None` (fully global CSS).
- **Class vs template:** class = state/behavior; template = declarative view bound one-way (property/interpolation) and event-driven (event binding) back into the class; expressions run in the component instance's context.
- **Host bindings/listeners:** target the *host element* (the matched tag itself), not the internal template; via `@HostBinding` / `@HostListener` decorators or the `host: {}` metadata object — both compile to the same `hostBindings` function.
- **Styling:** `:host`, `:host()`, `:host-context()` for component-scoped/conditional/ancestor-aware styling; `::ng-deep` (deprecated but working) to pierce emulated encapsulation; CSS custom properties cross boundaries cleanly and are the modern theming recommendation.
- **ng-content:** structural placeholder for consumer-supplied markup; unnamed slot = catch-all; `select="..."` = named slot by tag/attribute/class; projected content's bindings evaluate in the *parent's* context; projection is fixed at compile time, not dynamically re-orderable, and unmatched content with no catch-all slot is dropped silently.
- **Component API surface:** selector/inputs/outputs/exportAs (template compiler) · providers/viewProviders (DI) · host bindings/encapsulation/styles (DOM) · public class members (code-level `@ViewChild` / `@ContentChild` access).
- **Internals:** `@Component` compiles to a static `ɵcmp` definition (`defineComponent`) containing selectors, `decls` / `vars`, a shared `template()` render function (creation + update pass via render flags), `hostBindings()`, encapsulation/changeDetection flags. Runtime instances live in an `LView` (per-instance state) paired with a shared `TView` (per-type compiled structure) — this separation is why instantiating many copies of one component type is cheap.

Created By - Durgesh Singh